Project Report: Building and Analysing a Near-Real-Time Walmart Data Warehouse

Course: DS3003/DS3004: Data Warehousing & Business Intelligence

Semester: Fall 2025

1. Project Overview

This project's objective was to design, implement, and analyze a prototype of a near-real-time Data Warehouse (DW) for Walmart. The core task involved building a complete ETL (Extract, Transform, Load) pipeline to process a continuous stream of raw transactional data.

The system was designed to enrich this incomplete stream by joining it with master data (for customers, products, stores, and suppliers) in near-real-time. To accomplish this, the HYBRIDJOIN algorithm was implemented in Python, managing a "stream-to-relation" join.

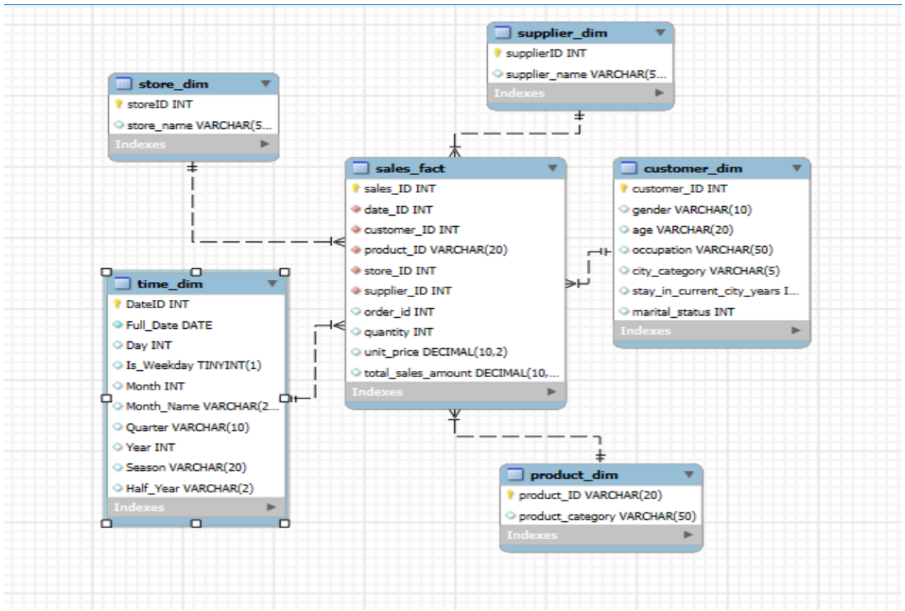
The final, enriched data was loaded into a relational star schema, which was then used for a wide range of OLAP (Online Analytical Processing) queries to derive business insights.

2. The Data Warehouse (DW) Schema

A classic **star schema** was designed to support the analytical requirements of the project. This design provides high query performance by separating the quantitative "facts" (like sales) from the descriptive "dimensions" (like customers or products).

The schema consists of one central fact table and five dimension tables:

- **sales_fact (Fact Table):** The central table. Each row represents a single sale transaction. It contains foreign keys to all dimensions and the core measures (quantity, unit_price, total_sales_amount).
- **customer_dim (Dimension):** Stores all customer-specific attributes, such as gender, age, occupation, and city_category.
- **product_dim (Dimension):** Stores product attributes, specifically product_category.
- **store_dim (Dimension):** Stores store information, such as store_name.
- **supplier_dim (Dimension):** Stores supplier information, such as supplier_name.
- **time_dim (Dimension):** A pre-calculated date dimension. It stores attributes for every single day, such as Is_Weekday, Month, Quarter, and Season, which is crucial for time-based analysis.



This schema is normalized (dimensions are split) and efficient, allowing for fast joins and aggregations to answer the 20 analytical queries.

3. Explanation of the HYBRIDJOIN Algorithm

The HYBRIDJOIN algorithm is designed to solve a specific problem: joining a fast, unbounded data stream (S) with a very large, disk-based relation (R) that does not fit in memory.

Our implementation followed the project requirements by using two independent threads:

1. **Thread 1: The Stream Feeder**
    - This thread's only job is to simulate the "near-real-time" stream.
    - It reads the transactional_data.csv file one row at a time.
    - It acquires a slot from a Semaphore (representing the 10,000 hash table slots).
    - It places the raw transaction row into a thread-safe stream_buffer.
2. **Thread 2: The Joiner (HYBRIDJOIN Logic)**
    - This thread runs the main algorithm in a loop.
    - **Step 1: Load Stream to Hash (H):** It pulls available transactions from the stream_buffer, hashes them on their Customer_ID, and stores them in an in-memory hash_table (a dictionary). It also adds the key to a fifo_queue (a deque) to track arrival order.
    - **Step 2: Load Relation (R):** It fetches a 500-tuple partition (the "disk buffer") from the customer_dim table (our Relation R).
    - **Step 3: Probe, Join, and Load:** It iterates through the 500-customer chunk. For each customer, it "probes" the hash_table using the customer_ID.

        - **On Match:** If a match is found, it performs the full data enrichment by looking up the product_master_data (in memory) and the time_dim (in the database). It then assembles the final row and loads it into the sales_fact table.
        - **On Miss:** If no match is found, it moves to the next customer in the partition.
    - **Step 4: Cleanup & Loop:** After a successful match, the transaction is removed from the hash_table and fifo_queue, and its slot is released back to the Semaphore. The thread then fetches the next 500-customer partition and repeats the process.

This two-thread model allows the data ingestion and the complex join/load process to run concurrently without blocking each other.

4. Three Shortcomings of HYBRIDJOIN

While effective, the HYBRIDJOIN algorithm as specified has several shortcomings:

1. **Vulnerability to Data Skew:** The algorithm's performance is highly dependent on the join key (Customer_ID). If a few customers (e.g., "super-users" or resellers) are responsible for a massive number of transactions, the hash_table will become "skewed." One bucket will contain thousands of entries, slowing down probes and potentially causing memory issues.
2. **Inefficient Queue Removal:** The algorithm requires matched tuples to be removed from the fifo_queue to prevent the queue from growing infinitely. In our Python implementation (using a deque), an O(N) operation is removing an item from the middle of the queue. This removal step becomes a large performance bottleneck as the queue starts to grow.
3. **Potential for Starvation/Stale Data:** The joiner thread iterates through the entire customer_dim relation in a loop. If this relation is massive (billions of customers), it could take a very long time to complete one full pass. A transaction from a customer at the *end* of the relation (e.g., Customer Z) might sit in the hash table for hours until the relation cursor finally reaches it, defeating the "near-real-time" objective.

5. Lessons Learned from the Project

This project was a comprehensive, hands-on journey through the entire lifecycle of a data warehouse.

The most important thing that was learned is that a DW is not just a database but an ETL system. The Create-DW.sql and Queries-DW.sql files were the "what," but the Hybrid-Join.py script was the "how." Actually building the Python script taught that the real challenge is in efficiently handling, cleaning, and transforming enormous datasets before they can even be analyzed.

Debugging the KeyErrors from the CSV files reinforced the importance of **data validation and contracts** in an ETL pipeline. A single mismatched column header (Date vs. date) can bring the entire system to a halt.

Finally, writing the 20 analysis queries provided a clear "Aha!" moment. I saw *why* the star schema is so powerful. Operations that would be complex in a normal database (like PARTITION BY, LAG(), and ROLLUP) were fast and intuitive because the schema was purpose-built for this kind of OLAP analysis. I learned to think in terms of measures, dimensions, and slices, which is the foundation of all business intelligence.